

AI ASSISTED CODING

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE DEPARTMENT
OF COMPUTER SCIENCE ENGINEERING

R . Aravind

2303A51215

BATCH – 04

16 – 01 – 2026

ASSIGNMENT – 2.5

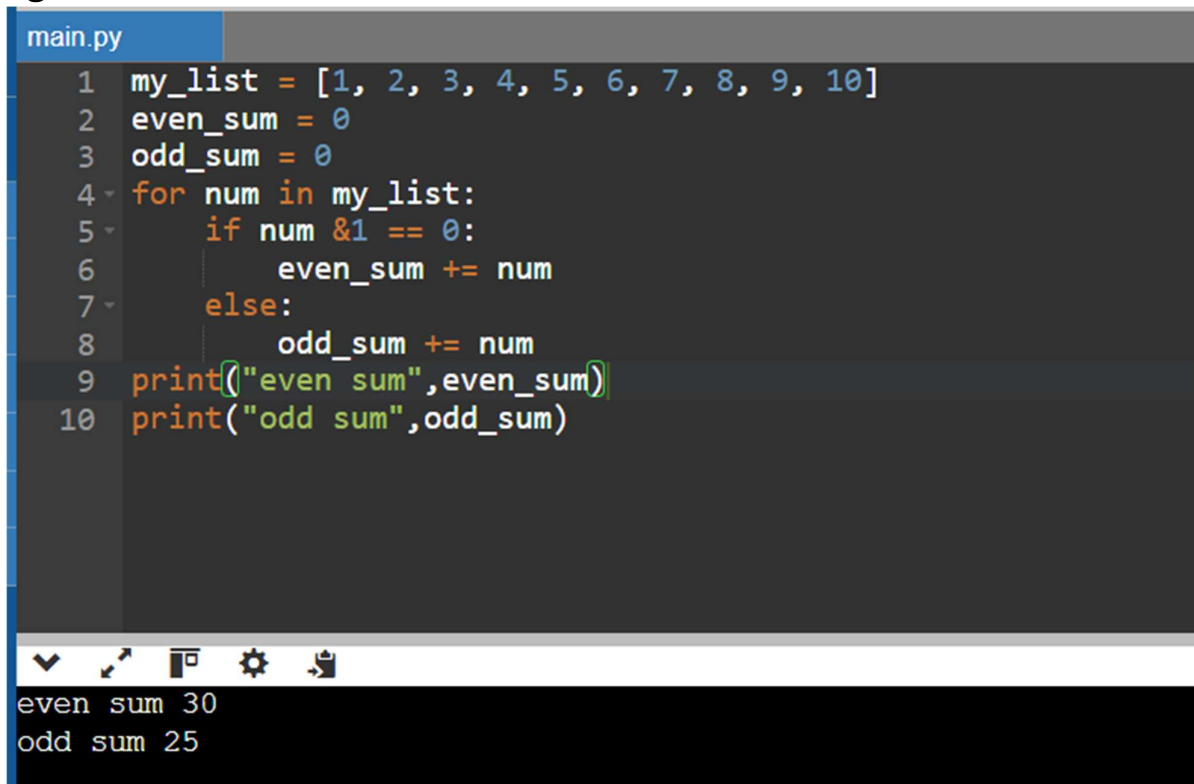
Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini and Cursor AI.

Task 1: Refactoring Odd/Even Logic (List Version).

Prompt: Write a Python program that takes a list of integers and calculates the sum of even numbers and the sum of odd numbers separately. After generating the working program, refactor (improve) the code using AI to make it cleaner, more readable, and efficient. Provide both the original code and the refactored version.

Code:

Original Code:



```
main.py
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 even_sum = 0
3 odd_sum = 0
4 for num in my_list:
5     if num &1 == 0:
6         even_sum += num
7     else:
8         odd_sum += num
9 print("even sum",even_sum)
10 print("odd sum",odd_sum)
```

even sum 30
odd sum 25

AI Improved Code:

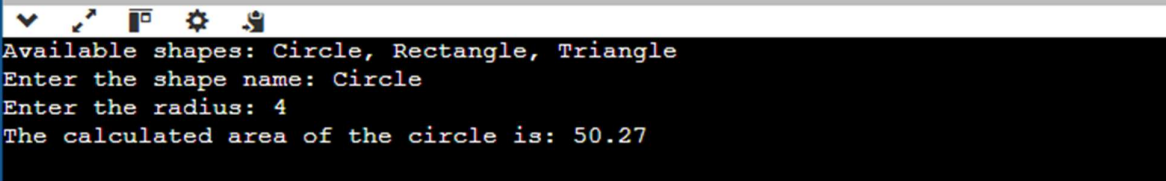
```
main.py
1 def calculate_sums(numbers):
2     even_sum = 0
3     odd_sum = 0
4
5     for num in numbers:
6         # Check if the number is divisible by 2
7         if num % 2 == 0:
8             even_sum += num
9         else:
10            odd_sum += num
11
12    return even_sum, odd_sum
13
14 # Example usage:
15 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
16 evens, odds = calculate_sums(my_list)
17
18 print(f"List: {my_list}")
19 print(f"Sum of even numbers: {evens}")
20 print(f"Sum of odd numbers: {odds}")
```

List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Sum of even numbers: 30
Sum of odd numbers: 25

Task 2: Area Calculation Explanation.

Prompt: Write a Python function that calculates the area of different shapes (circle, square, and rectangle) based on user input. Then explain the function line by line in simple language so that a junior developer can easily understand it. **Code:**

```
main.py
1 import math
2 def calculate_area(shape, dims):
3     if shape == "circle":
4         return math.pi * (dims[0] ** 2)
5     elif shape == "rectangle":
6         return dims[0] * dims[1]
7     elif shape == "triangle":
8         return 0.5 * dims[0] * dims[1]
9     return None
10 # --- User Input Section ---
11 print("Available shapes: Circle, Rectangle, Triangle")
12 shape_choice = input("Enter the shape name: ").strip().lower()
13
14 if shape_choice == "circle":
15     r = float(input("Enter the radius: "))
16     area = calculate_area(shape_choice, [r])
17 elif shape_choice in ["rectangle", "triangle"]:
18     # These both need two values: (length/width) or (base/height)
19     val1 = float(input("Enter the first dimension (length/base): "))
20     val2 = float(input("Enter the second dimension (width/height): "))
21     area = calculate_area(shape_choice, [val1, val2])
22
23 else:
24     area = None
25     print("Invalid shape selection.")
26
27 if area is not None:
28     print(f"The calculated area of the {shape_choice} is: {area:.2f}")
```



```
Available shapes: Circle, Rectangle, Triangle
Enter the shape name: Circle
Enter the radius: 4
The calculated area of the circle is: 50.27
```

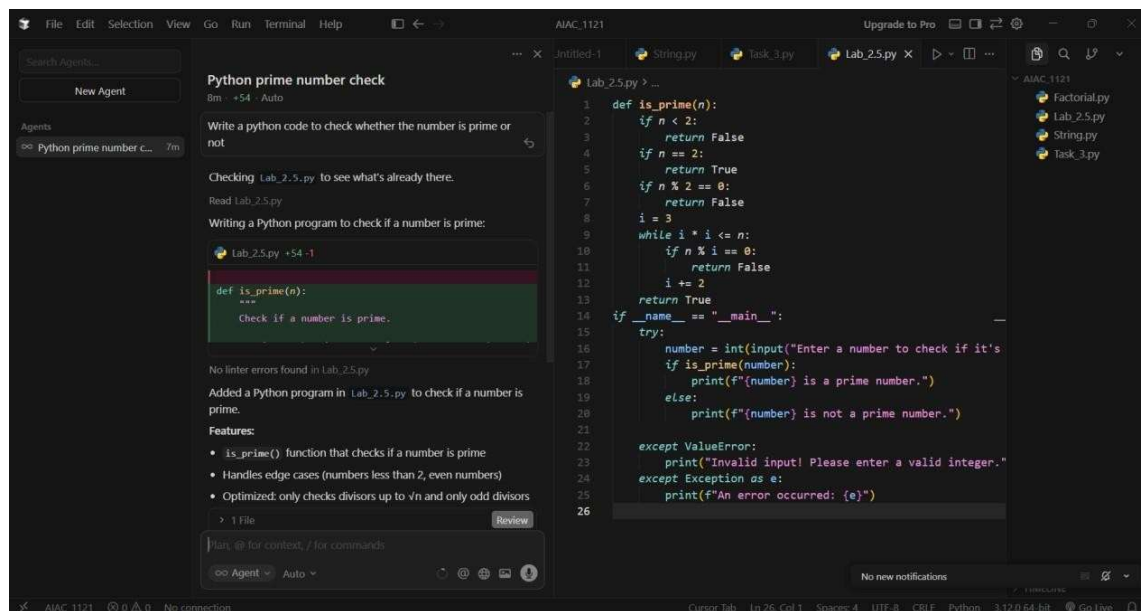
Explanation:

The `calculate_area()` function helps users find the area of circles, squares, or rectangles. It asks for the shape and necessary dimensions, like radius or side

length, then calculates and prints the result. The function includes error handling to manage invalid inputs and ensures dimensions are non-negative, using Python's math module for calculations.

Task 3: Prompt Sensitivity Experiment.

Prompt 1(Simple): Write a Python Code to check whether the given Number is Prime or Not **Code:**

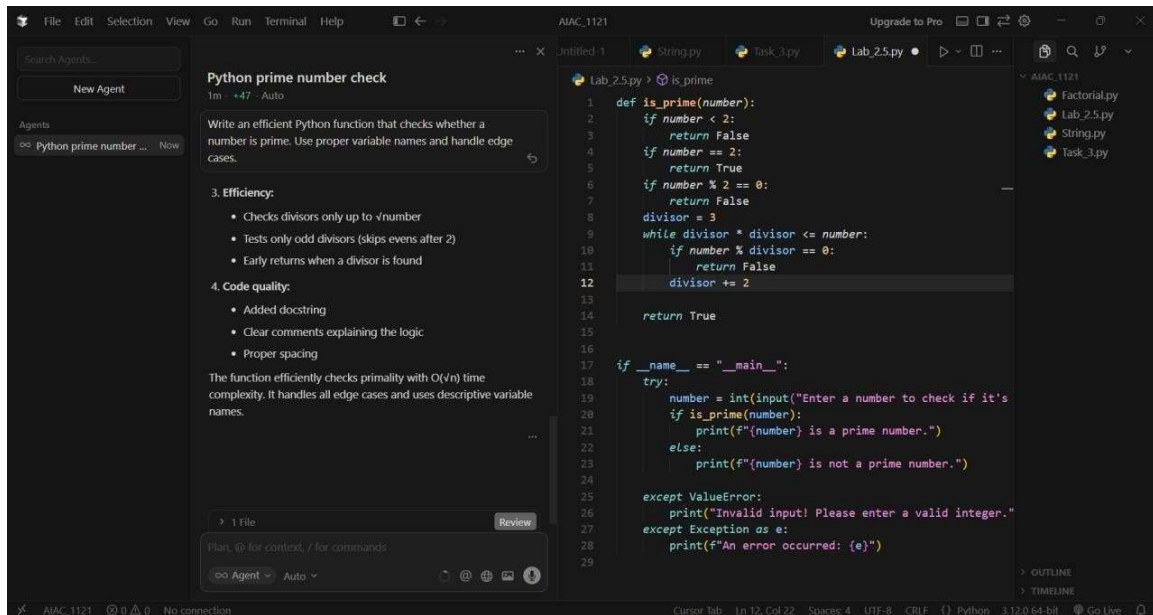


The screenshot shows a code editor with a file named 'Lab_2.5.py'. The code defines a function 'is_prime(n)' that checks if a number is prime. It includes error handling for invalid inputs and edge cases. The function returns True for prime numbers and False otherwise. The code is as follows:

```
1 def is_prime(n):
2     if n < 2:
3         return False
4     if n == 2:
5         return True
6     if n % 2 == 0:
7         return False
8     i = 3
9     while i * i <= n:
10        if n % i == 0:
11            return False
12        i += 2
13    return True
14 if __name__ == "__main__":
15     try:
16         number = int(input("Enter a number to check if it's
17         if is_prime(number):
18             print(f"{number} is a prime number.")
19         else:
20             print(f"{number} is not a prime number.")
21     except ValueError:
22         print("Invalid input! Please enter a valid integer.")
23     except Exception as e:
24         print(f"An error occurred: {e}")
25
26
```

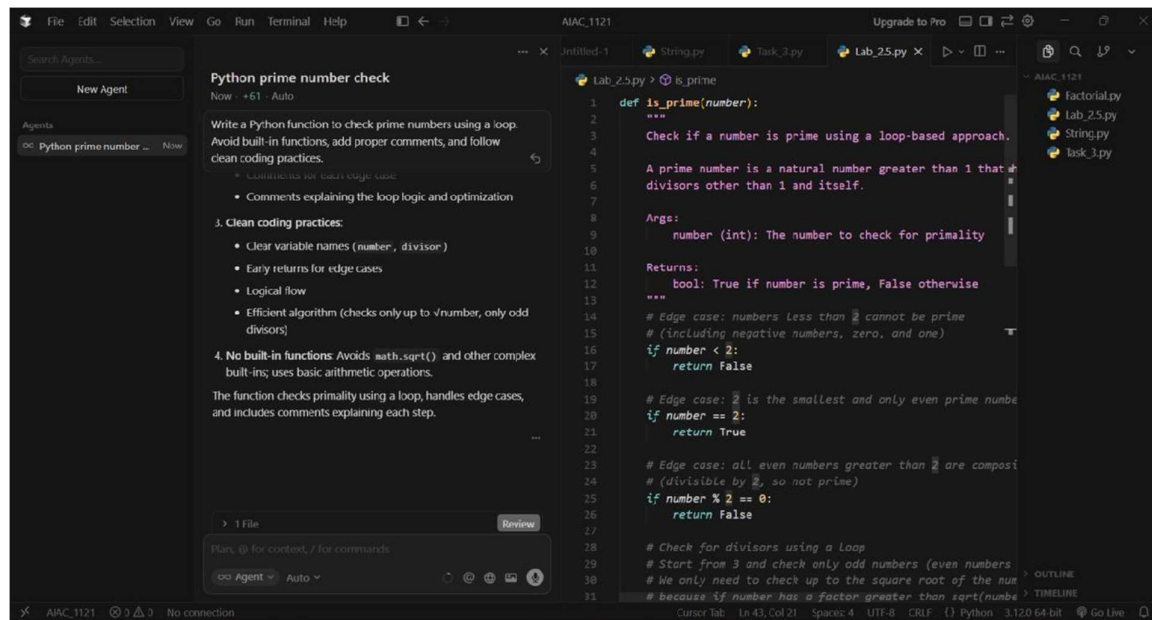
Prompt 2(Medium & Detailed): Write an efficient Python function that checks whether a number is prime. Use proper variable names and handle edge cases.

Code:



Prompt 3(Hard & More Detailed): Write a Python function to check prime numbers using a loop. Avoid built-in functions, add proper comments, and follow clean coding practices.

Code:



Task 4: Tool Comparison Reflection.

Prompt: Based on my experiments with Gemini, GitHub Copilot, and Cursor AI, compare these three tools in terms of:

- Ease of use
- Code quality
- Clarity of explanation
- Overall usefulness for students and developers Write a short reflection (8–10 lines).

Short Reflection:

Based on my experience, all three AI tools—Gemini, GitHub Copilot, and Cursor AI—are useful for AI-assisted coding, but in different ways. Gemini was very helpful for understanding concepts because it provided clear explanations along with code. GitHub Copilot generated concise and professional-level code, making it suitable for real-world development. However, Copilot gave limited explanations compared to Gemini. Cursor AI was useful for experimenting with different prompts and observing how code changes based on instructions. It also helped in refactoring and improving existing code effectively. In terms of usability, Gemini was the most beginnerfriendly, while Copilot and Cursor were better suited for experienced programmers. Overall, Gemini is best for learning, Copilot for coding speed, and Cursor AI for refinement and experimentation.